

InvadersHunter

A Web Development Tutorial Through Your Own Code

This document explains how your full-stack application works — file by file, layer by layer — from a web development learning perspective.

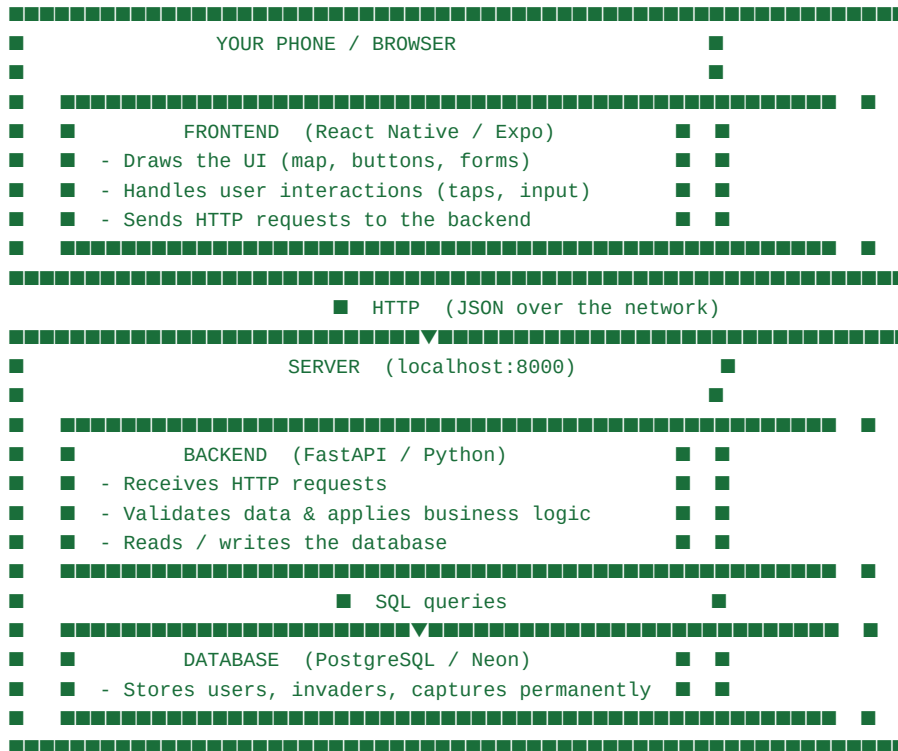
Stack: Python · FastAPI · PostgreSQL · React Native · Expo · TypeScript

Table of Contents

- 1 The Big Picture — What is a Full-Stack App?
- 2 The Tech Stack
- 3 The Backend — FastAPI + Python
 - 3.1 What is a Backend?
 - 3.2 Entry Point: `main.py`
 - 3.3 The Database: `database.py`
 - 3.4 Models: The Shape of Your Data
 - 3.5 Schemas: Validating Data In and Out
 - 3.6 Routers: The API Endpoints
 - 3.7 Security: Passwords and JWT Tokens
 - 3.8 Email: The `email.py` Service
 - 3.9 Dependencies: `dependencies.py`
- 4 The Frontend — React Native + Expo
 - 4.1 What is a Frontend?
 - 4.2 React and Components
 - 4.3 TypeScript: Why `.tsx` Instead of `.js`
 - 4.4 File-Based Routing with Expo Router
 - 4.5 The Root Layout: `_layout.tsx`
 - 4.6 The Tab Layout
 - 4.7 Screens: `map`, `login`, `register`, `forgot-password`
 - 4.8 State Management with Zustand: `store.ts`
 - 4.9 The API Client: `api-client.ts`
 - 4.10 Feature Services: `auth.api.ts` & `invaders.api.ts`
 - 4.11 Types and Mappers
 - 4.12 Map Components
- 5 How Frontend and Backend Talk to Each Other
 - 5.1 HTTP and REST
 - 5.2 The Full API Endpoint Table
 - 5.3 Authentication Flow Step by Step
 - 5.4 A Request's Full Journey: Flashing an Invader
- 6 The Database Schema
- 7 Project Configuration Files
- 8 Key Web Development Concepts

1. The Big Picture

An app like InvadersHunter is split into three layers that each have a clear responsibility and only communicate with their direct neighbor:



The frontend never touches the database directly — it always goes through the backend. This separation keeps security, logic, and presentation cleanly isolated.

2. The Tech Stack

Layer	Technology	Language	What it does
Frontend	React Native	TypeScript	Builds the UI with components
Frontend	Expo	TypeScript	Cross-platform build tool (iOS, Android, Web)
Frontend	Expo Router	TypeScript	Maps file paths to app screens
Frontend	Zustand	TypeScript	Stores global state (auth token, user)
Frontend	Axios	TypeScript	Sends HTTP requests to the backend
Frontend	MapLibre GL	TypeScript	Renders the interactive map
Backend	FastAPI	Python	HTTP server that handles API requests
Backend	SQLAlchemy	Python	Talks to the database using Python objects
Backend	Pydantic	Python	Validates and serializes data
Backend	python-jose	Python	Creates and verifies JWT auth tokens
Backend	bcrypt	Python	Hashes passwords securely
Backend	FastAPI-Mail	Python	Sends emails (password reset)
Backend	Alembic	Python	Manages database schema migrations
Database	PostgreSQL	SQL	Relational database
Hosting	Neon	—	Cloud PostgreSQL provider

3. The Backend — FastAPI + Python

3.1 What is a Backend?

The backend is a program that runs on a server. It listens for HTTP requests (like "give me the list of invaders"), processes them, and sends back HTTP responses (like "here are the invaders in JSON format"). In this project the backend lives in the [backend/](#) folder.

3.2 Entry Point: main.py

File: backend/app/main.py — Language: Python

This is the first file that runs when you start the backend server. It:

- Creates the FastAPI application object
- Configures CORS (which frontends are allowed to call the API)
- Registers all the routers (groups of API endpoints)

Simplified version of what main.py does:

```
app = FastAPI()                                # 1. Create the app

app.add_middleware(CORSMiddleware,             # 2. Allow the frontend to call the API
    allow_origins=["*"],                        #   "*" = any origin (fine for dev, not prod)
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth_router)                 # 3. Register route groups
app.include_router(users_router)
app.include_router(invaders_router)
app.include_router(progress_router)
```

Key concept — CORS: Browsers block JavaScript from calling a different domain unless the server explicitly allows it. The CORSMiddleware adds [Access-Control-Allow-Origin](#) headers to responses so the browser permits the call.

To start the server you run: `uvicorn app.main:app --reload` — uvicorn is the web server that hosts your FastAPI app on port 8000.

3.3 The Database: database.py

File: backend/app/database.py — Language: Python

This file sets up the connection between Python and PostgreSQL using SQLAlchemy.

```
DATABASE_URL = "postgresql://..."           # Address of the Neon cloud database

engine        = create_engine(DATABASE_URL)    # The connection pool to the DB
SessionLocal  = sessionmaker(engine)          # Factory to create DB sessions
Base           = declarative_base()            # Parent class for all DB models
```

- **engine:** The persistent connection to PostgreSQL.
- **Session:** A unit of work — open it, do reads/writes, commit, close.
- **Base:** Every model class inherits from it; SQLAlchemy creates a table for each.

3.4 Models: The Shape of Your Data

Files: backend/app/models/user.py · space_invader.py · user_progress.py

Models are Python classes that map directly to database tables. SQLAlchemy reads these classes and creates (or verifies) the corresponding tables.

```
# user.py – simplified
class User(Base):
    __tablename__ = "users"          # Name of the SQL table

    id = Column(Integer, primary_key=True)
    username = Column(String, unique=True)
    email = Column(String, unique=True)
    hashed_password = Column(String)
    created_at = Column(DateTime, default=datetime.utcnow)
    is_admin = Column(Boolean, default=False)
```

The three models and their relationship:

```
users ■■■■■■■■■■■■■■■■■■■■■■ user_progress ■■■■■■■■■■■■■■■■■■■■■■ invaders
    id ■■■ user_id (FK)                invader_id (FK) ■■■ id
```

user_progress is a JOIN TABLE: it links a user to an invader with a timestamp.
This implements a many-to-many relationship (one user can find many invaders,
one invader can be found by many users).

3.5 Schemas: Validating What Comes In and Goes Out

Files: backend/app/schemas/*.py — Language: Python (Pydantic)

While **models** define the database structure, **schemas** define the shape of data coming into and going out of the API.

```
class UserCreate(BaseModel):          # What the frontend sends at registration
    username: str
    email: EmailStr                    # Pydantic validates this is a real email
    password: str

class UserOut(BaseModel):              # What the backend sends back (NEVER the password!)
    id: int
    username: str
    email: str
    created_at: datetime

class Config:
    from_attributes = True             # Allows converting a SQLAlchemy model to this schema
```

If the frontend sends `"email": "not-an-email"`, Pydantic rejects it with a 422 error before any business logic runs.

3.6 Routers: The API Endpoints

Files: backend/app/api/routers/*.py

Routers group related endpoints together. Each function decorated with `@router.get(...)` or `@router.post(...)` becomes one URL the frontend can call.

```
# auth.py – simplified
router = APIRouter(prefix="/auth", tags=["auth"])

@router.post("/login")
def login(request: LoginRequest, db: Session = Depends(get_db)):
    # 1. Find user by username
```

- `@router.post("/login")` → handles POST `http://localhost:8000/auth/login`
- `request: LoginRequest` → Pydantic validates the incoming JSON body automatically
- `db: Session = Depends(get_db)` → FastAPI injects a database session
- `raise HTTPException(401)` → sends an HTTP 401 error to the frontend

3.9 Dependencies: dependencies.py

File: backend/app/dependencies.py

```
def get_db():
    db = SessionLocal()    # Open a DB session
    try:
        yield db           # Pass it to the endpoint function
    finally:
        db.close()         # Always close it, even on error
```

Every endpoint that needs database access declares `db: Session = Depends(get_db)` and FastAPI automatically calls this generator, passes the session, and closes it after. This pattern ensures DB connections are never leaked.

4. The Frontend — React Native + Expo

4.1 What is a Frontend?

The frontend is the code that runs on the user's device (phone or browser). It draws the interface, handles user interactions, and communicates with the backend. In this project the frontend lives in [frontend/src/](#).

4.2 React and Components

React is a JavaScript library for building UIs. The key idea is **components**: small, reusable pieces of UI defined as functions.

```
// A simple React component
function Greeting({ name }: { name: string }) {
  return <Text>Hello, {name}!</Text>;
}

// Used like an HTML tag:
<Greeting name="Alice" />
```

React uses **JSX** (or TSX for TypeScript): a syntax that looks like HTML but is actually JavaScript/TypeScript. When data changes (like the user logs in), React **automatically re-renders** only the components that depend on that data.

4.3 TypeScript: Why .tsx Instead of .js

```
// Plain JavaScript – what are a and b? Strings? Numbers?
function add(a, b) { return a + b; }

// TypeScript – explicit and safe
function add(a: number, b: number): number { return a + b; }
```

- Catch bugs at compile time, not at runtime
- Better autocomplete in your editor (IntelliSense)
- Self-documenting code — types describe the data
- **.tsx** = TypeScript + JSX (TypeScript files containing React components)

4.4 File-Based Routing with Expo Router

Files in: `frontend/src/app/`

Expo Router turns **file paths** into **app screens** automatically. You don't define routes in code — the file system IS the router.

```
frontend/src/app/
■■■■ _layout.tsx      → Root layout (wraps everything)
■■■■ index.tsx        → "/" screen (redirects to map)
■■■■ login.tsx        → "/login" screen
■■■■ register.tsx     → "/register" screen
■■■■ forgot-password.tsx → "/forgot-password" screen
■■■■ (tabs)/
  ■■■■ _layout.tsx    → Tab bar layout
  ■■■■ map.tsx        → "/map" tab
  ■■■■ invader.tsx    → "/invader" tab
  ■■■■ profile.tsx    → "/profile" tab
```

The **(tabs)** folder name with parentheses is a **route group** — it organizes files without adding to the URL. Files inside share the tab bar layout.

```
import { router } from 'expo-router';
router.push('/login');    // Navigate to login screen
router.replace('/map');   // Navigate and remove current screen from history
```

4.5 The Root Layout: `_layout.tsx`

File: frontend/src/app/_layout.tsx

The outermost wrapper of the entire app. Its main job is **auth routing** — protecting private screens from unauthenticated users.

```
export default function RootLayout() {
  const token = useAuthStore((s) => s.token); // Read token from Zustand
  const segments = useSegments();             // Current URL segments

  useEffect(() => {
    const isPublic = ['login', 'register', 'forgot-password'].includes(segments[0]);

    if (!token && !isPublic) {
      router.replace('/login'); // Not logged in? Go to login
    } else if (token && isPublic) {
      router.replace('/map');   // Already logged in? Go to map
    }
  }, [token, segments]); // Re-runs whenever token or route changes

  return <Stack />;       // Render the active screen
}
```

4.6 The Tab Layout

File: frontend/src/app/(tabs)/_layout.tsx

Defines the bottom tab bar shown on all authenticated screens — the navigation bar with icons for Map, Invaders, and Profile tabs.

4.7 Screens

map.tsx — The Main Screen

File: frontend/src/app/(tabs)/map.tsx

- Fetches all invaders and the user's captures from the API
- Combines them with `mapInvadersWithProgress` to know which are already flashed
- Passes everything to the `WebMap` component for display
- Handles flash/unflash actions and updates state

login.tsx — Login Screen

File: frontend/src/app/login.tsx

A form with username and password. On submit, calls `loginUser()` → the auth store saves the token → the root layout detects it and redirects to `/map`.

forgot-password.tsx — 3-Step Password Reset

File: frontend/src/app/forgot-password.tsx

A single screen with 3 steps controlled by a local step state variable:

- **Step 1:** Enter username + email → calls forgotPassword()
- **Step 2:** Enter 6-digit code received by email → calls verifyResetCode()
- **Step 3:** Enter new password → calls resetPassword()

4.8 State Management with Zustand: store.ts

File: frontend/src/features/auth/store.ts

State management = a way to store data that multiple components need to share. Zustand is a lightweight library for this.

```
const useAuthStore = create<AuthStore>((set) => ({
  token: null,    // The JWT token (null = not logged in)
  user: null,     // Parsed user info (id, is_admin, ...)

  login: (token) => {
    const decoded = parseJwt(token); // Decode JWT to get user info
    set({ token, user: decoded });
  },

  logout: () => set({ token: null, user: null }),
}));

// In any component:
const token      = useAuthStore((s) => s.token);
const { login }  = useAuthStore();
```

Why not use a regular variable? React components would not know when it changed and would not re-render. Zustand's store is **reactive** — any component that reads from it re-renders automatically when it changes.

4.9 The API Client: api-client.ts

File: frontend/src/services/api-client.ts

Creates a configured **Axios instance** that all API calls use.

```
const apiClient = axios.create({
  baseURL: 'http://127.0.0.1:8000', // All calls go to this server
});

// Interceptor: runs automatically before EVERY request
apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`; // Add auth header
  }
  return config;
});
```

Instead of manually adding the auth token to every API call, the interceptor does it automatically for all requests.

4.10 Feature Services

Files: frontend/src/features/auth/services/auth.api.ts · invaders/services/invaders.api.ts

These files contain functions that call specific API endpoints. They are the **only place** in the frontend that knows which URLs exist on the backend.

```
// auth.api.ts
```

```

export async function loginUser(username: string, password: string) {
  const res = await apiClient.post('/auth/login', { username, password });
  return res.data;    // { access_token: "...", token_type: "bearer" }
}

// invaders.api.ts
export async function flashInvader(userId: number, invaderId: number) {
  const res = await apiClient.post('/progress/', { user_id: userId, invader_id: invaderId });
  return res.data;
}

```

Why separate these from components? This is **separation of concerns**: components handle UI, services handle data fetching. If a URL changes you only update the service file.

4.11 Types and Mappers

Files: frontend/src/features/invaders/types.ts · mapper.ts

```

// types.ts
type Invader = { id: number; name: string; latitude: number; longitude: number; points: number | null; };
type Capture = { id: number; user_id: number; invader_id: number; found_at: string; };
type InvaderWithState = Invader & { isFlashed: boolean; captureId?: number; };

// mapper.ts – merges two lists from separate API calls
function mapInvadersWithProgress(invaders: Invader[], captures: Capture[]): InvaderWithState[] {
  return invaders.map(inv => {
    const capture = captures.find(c => c.invader_id === inv.id);
    return { ...inv, isFlashed: !!capture, captureId: capture?.id };
  });
}

```

4.12 Map Components

File	Role
web-map.tsx	MapLibre GL map, places a Marker for each invader, opens popup on click
invader-marker.tsx	Custom pin graphic, changes appearance based on isFlashed state
invader-popup.tsx	Info card showing name/points + Flash or Unflash button

5. How Frontend and Backend Talk to Each Other

5.1 HTTP and REST

The frontend and backend communicate using **HTTP** — the same protocol your browser uses to load web pages.

Each HTTP message has:

- **Method:** GET (read), POST (create), PUT (update), DELETE (remove)
- **URL:** the address of the resource (e.g., /invaders/4)
- **Headers:** metadata (e.g., Authorization: Bearer <token>)
- **Body:** the data sent — for POST/PUT, usually JSON

REST is a convention for designing these URLs:

Goal	Method	URL
Get all invaders	GET	/invaders/
Get one invader	GET	/invaders/4
Create an invader	POST	/invaders/
Update an invader	PUT	/invaders/4
Delete an invader	DELETE	/invaders/4

The backend responds with a **status code**:

Code	Meaning
200 OK	Success
201 Created	Resource was created
401 Unauthorized	You need to log in
404 Not Found	Resource doesn't exist
422 Unprocessable	Request data failed validation

5.2 The Full API Endpoint Table

Method	Endpoint	Auth?	Purpose
POST	/auth/login	No	Log in, get JWT token
POST	/auth/forgot-password	No	Request password reset code
POST	/auth/verify-reset-code	No	Verify the 6-digit code
POST	/auth/reset-password	No	Set new password
GET	/users/	Yes	List all users
POST	/users/	No	Register new user
PUT	/users/{user_id}	Yes	Update user profile

Method	Endpoint	Auth?	Purpose
DELETE	/users/{user_id}	Yes	Delete user
GET	/invaders/	Yes	Get all invaders with GPS coords
POST	/invaders/	Yes	Create invader
PUT	/invaders/{id}	Yes	Update invader
DELETE	/invaders/{id}	Yes	Delete invader
GET	/progress/	Yes	Get all captures
GET	/progress/user/{user_id}	Yes	Get a user's captures
POST	/progress/	Yes	Record a capture (flash)
PUT	/progress/{id}	Yes	Update a capture
DELETE	/progress/{id}	Yes	Remove a capture (unflash)

5.3 Authentication Flow Step by Step

Registration

1. User fills form → frontend calls POST /users/ with {username, email, password}
2. Backend validates the data (Pydantic schema)
3. Backend checks username/email not already taken (DB query)
4. Backend hashes the password with bcrypt
5. Backend inserts User row into the database
6. Backend sends welcome email
7. Backend returns the new user data
8. Frontend auto-logs in the user (creates a JWT)
9. Frontend stores token in Zustand
10. Root layout detects token → redirects to /map

Login

1. User submits username + password
2. Frontend calls POST /auth/login
3. Backend finds user by username (DB query)
4. Backend verifies password with bcrypt.checkpw()
5. If valid: backend creates JWT {sub: user_id, is_admin: false, exp: +30d}
6. Backend returns {access_token: "eyJ..."}
7. Zustand store.login() saves the token and parses user ID
8. Root layout detects token → redirects to /map

Subsequent Requests

1. Frontend calls any protected endpoint (e.g., GET /invaders/)
2. Axios interceptor adds "Authorization: Bearer eyJ..." automatically
3. Backend receives request, decodes JWT, identifies user
4. Backend processes request and returns data

5.4 A Request's Full Journey: Flashing an Invader

Let's trace exactly what happens when a user taps **Flash** on an invader popup:

```
[invader-popup.tsx]
  User taps "Flash" button
    ↓ onFlash() callback called
[map.tsx]
```

```
handleFlash(invaderId) called
↓
[invaders.api.ts]
  flashInvader(userId, invaderId)
↓
[api-client.ts]
  axios.post('/progress/', {user_id: 4, invader_id: 7})
  + Interceptor adds header: "Authorization: Bearer eyJ..."
↓
  HTTP POST request travels over the network
↓
[main.py]
  FastAPI receives the request, routes it to progress router
↓
[user_progress.py router]
  @router.post("/progress/") runs
  Pydantic validates the body
  DB: INSERT INTO user_progress (user_id, invader_id, found_at) VALUES (4, 7, now())
  Returns new UserProgress as JSON
↓
[map.tsx]
  Response received → state updated → invader shown as captured
  React re-renders the marker with the flashed style
```

6. The Database Schema

```
-- Users table
CREATE TABLE users (
    id            SERIAL PRIMARY KEY,
    username      VARCHAR UNIQUE NOT NULL,
    email         VARCHAR UNIQUE NOT NULL,
    hashed_password VARCHAR NOT NULL,
    created_at    TIMESTAMP DEFAULT NOW(),
    is_admin      BOOLEAN DEFAULT FALSE
);

-- Invaders table
CREATE TABLE invaders (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR NOT NULL,
    image_url     VARCHAR,
    description   VARCHAR,
    points        INTEGER,
    state         VARCHAR,
    latitude      FLOAT,
    longitude     FLOAT
);

-- Captures table (join table between users and invaders)
CREATE TABLE user_progress (
    id            SERIAL PRIMARY KEY,
    user_id       INTEGER REFERENCES users(id),
    invader_id    INTEGER REFERENCES invaders(id),
    found_at      TIMESTAMP DEFAULT NOW()
);
```

Alembic manages changes to this schema over time. When you add a new column or table you create an Alembic migration instead of manually running SQL — this lets the whole team apply the same DB changes safely.

7. Project Configuration Files

Backend

File	Purpose
backend/requirements.txt	Lists Python packages to install (pip install -r requirements.txt)
backend/.env	Secret env vars (DB URL, JWT secret, email creds). NEVER commit this!
backend/.env.example	Template showing which env vars are needed, with placeholder values

Frontend

File	Purpose
frontend/package.json	Lists JS packages and scripts (npm install, npm start)
frontend/app.json	Expo config: app name, icon, splash screen, plugins
frontend/tsconfig.json	TypeScript compiler options; defines @/* path alias → src/*

8. Key Web Development Concepts

REST API

A standard convention for designing URLs for data operations. GET = read, POST = create, PUT = update, DELETE = remove.

JSON

JavaScript Object Notation — the data format exchanged over HTTP. Both Python (via Pydantic) and TypeScript (native) understand it natively.

JWT (JSON Web Token)

A signed token proving a user's identity. Sent in the Authorization header of every request after login. Carries user ID and role.

ORM (Object-Relational Mapping)

SQLAlchemy maps Python classes to database tables. You write Python objects and it generates SQL queries automatically.

Dependency Injection

FastAPI's Depends() system injects shared objects (like the DB session) into endpoints automatically, ensuring proper lifecycle management.

Component-Based UI

React builds UIs from small, reusable functions. State changes trigger automatic, efficient re-renders of only the affected components.

State Management

Zustand stores global data (the auth token) that multiple components need. It is reactive — readers re-render when the state changes.

Interceptors

Axios interceptors run code before or after every HTTP request — used here to inject the auth header automatically on all calls.

File-Based Routing

Expo Router maps file paths directly to screens. No manual route configuration needed — create a file, get a route.

CORS

A browser security mechanism. Your backend must explicitly declare which frontend origins are allowed to call it.

Environment Variables

Secrets (DB passwords, API keys) are stored in .env files, not in code. They are loaded at runtime and ignored by git.

Password Hashing

bcrypt produces a one-way hash of a password. It can be verified but never reversed — even if your database is compromised, passwords stay safe.

Generated from the InvadersHunter codebase for educational purposes.